DevOps Engineer — Hands-On Assessment

Oct 12, 2026 · @Cloudly DevOps

Overview

This 6-hour, local-only assessment turns a simple React + Node.js + PostgreSQL app into a production-style platform using Docker, Kubernetes and optional Terraform.

Item	Detail
Duration	1 day
Environment	Local (Docker + Kubernetes via kind or minikube + optional Terraform)
Base repository	cloudly-io/node-react-postgres-platform, or any repository you are comfortable with

You are judged not only on whether it runs, but on how you:

- Design deployments and infrastructure
- Containerize and orchestrate services
- Configure networking and service-to-service communication
- Plan CI/CD and multi-environment workflows
- Integrate platform services (Redis, queues, search)
- Set up observability, autoscaling and rate limiting
- Document and explain your decisions

Task 1 — Local Containerized Setup (Simple)

Create a fully containerized local environment with Docker Compose, started by one command such as `docker compose up`.

Files to create

- `Dockerfile.backend`
- `Dockerfile.frontend`
- `.dockerignore`

- `docker-compose.yml`

Requirements

- Use multi-stage builds in both Dockerfiles to keep images small.
- Compose runs a backend service, a frontend service, and a PostgreSQL service with a persistent volume (data must survive container restarts).
- Use environment variables for DB host, port, user, password, DB name, and the API base URL the frontend uses to reach the backend.
- The app works end to end: the frontend is reachable in a browser, calls the backend, and the backend reads and writes Postgres.

Evaluation focus

- Clean, efficient Dockerfiles (multi-stage, minimal layers)
- Correct networking between services inside Docker
- No hardcoded secrets; proper use of environment variables
- Simple, well-documented local startup in the README

Task 2 — Kubernetes Deployment

Deploy the app into a local Kubernetes cluster (kind or minikube), with every resource in a `dev` namespace.

Suggested `k8s/` structure

Area	Files
Namespace	`namespace-dev.yaml`
Backend	`backend-deployment.yaml`, `backend-service.yaml`
Frontend	`frontend-deployment.yaml`, `frontend-service.yaml`
PostgreSQL	`postgres-deployment.yaml` or `postgres-statefulset.yaml`, plus `postgres-service.yaml`
Storage	`postgres-pv.yaml`, `postgres-pvc.yaml`
Configuration	`app-configmap.yaml`, `app-secrets.yaml`

Requirements

- All resources are created under the `dev` namespace.
- Backend and Postgres are exposed internally with ClusterIP services.
- The frontend is exposed externally with NodePort or Ingress.
- PostgreSQL uses a PersistentVolumeClaim for data persistence.
- The backend deployment has readiness and liveness probes.
- The full application works in the cluster: Frontend → Backend → DB.

Evaluation focus

- Kubernetes basics: Deployments, Services, ConfigMaps, Secrets, PV/PVC, Namespaces
- Clear separation of internal and external services
- Proper health checking through probes
- Logical, readable YAML structure and naming conventions

Task 3 — API Gateway / Ingress & Service Communication

Use an Ingress Controller as an API gateway to unify external access to the app.

Requirements

- Install and configure an Ingress Controller (for example, nginx-ingress) in the local cluster.
- Create an Ingress manifest such as `ingress-dev.yaml`.
- Route `/` to `frontend-service`, and `/api` or `/api/*` to `backend-service`.
- Make the app reachable at a friendly hostname such as `http://node-react.local`, using your local hosts file.

Document in the README

1. The full request path: Browser → Ingress → Service → Pod → PostgreSQL.
2. How Kubernetes DNS resolves services, for example `backend-service.dev.svc.cluster.local`.
3. How you would debug each case: (a) Ingress is reachable but the backend is not; (b) the frontend cannot talk to the backend.
4. A brief note on how this design could evolve into a microservice setup with multiple backend services behind an API gateway.

Evaluation focus

- Clear model of Ingress vs Service vs Pod
- Understanding of internal DNS and service discovery
- Ability to reason about traffic flow and debug network issues
- Design awareness for future microservice expansion

Task 4 — CI/CD & Multi-Environment Deployment

Design a CI/CD workflow that builds, tests and deploys to different Kubernetes namespaces based on branch name. The expected file is `.github/workflows/ci-cd.yml`, or an equivalent for Bitbucket or Jenkins with a similar structure.

Branch rules

Branch	Target namespace
`master`	`dev`
`pre-release/v*`	`preprod`
`release/v*`	`release`

The pipeline should

- Install frontend and backend dependencies.
- Build both services and run tests, or at least build validations.
- Build Docker images for the frontend and backend.
- Tag images with the branch name plus the short commit SHA.
- Push images to a container registry (Docker Hub, GHCR, etc.).
- Deploy images or manifests to the namespace that matches the branch.

Document in the README

- How CI secrets are stored and used (registry credentials, kubeconfig, etc.).
- A rollback strategy: previous image tags, or `kubectl rollout undo` on deployments.

Evaluation focus

- Realistic, structured CI/CD design
- Clear mapping between branches and environments

- Security awareness in handling secrets
- Understanding of safe deployment and rollback patterns

Task 5 — Platform Services: Redis, Queue, Search, Read/Write DB (Hard)

Expand the system with services typical of production platforms, in both Docker Compose and Kubernetes.

Suggested additional `k8s/` manifests

Service	Files
Redis	`redis-deployment.yaml`, `redis-service.yaml`
Queue (choose one)	BullMQ: `bullmq-worker-deployment.yaml` (uses Redis); or RabbitMQ: `rabbitmq-deployment.yaml`, `rabbitmq-service.yaml`
Search (Elasticsearch or OpenSearch)	`search-deployment.yaml`, `search-service.yaml`
Database cluster simulation	Env vars `DB_WRITE_URL` (primary) and `DB_READ_URL` (replica)

Requirements

- Add a Redis service, a queue service (BullMQ via Redis, or RabbitMQ), and a search service in dev mode.
- Wire the backend to them at least through configuration: it can reach Redis, it or a separate worker can connect to the queue, and it can reach the search engine (even only for a health call).
- Simulate a read/write DB cluster with separate write and read URLs in the app environment.

Document in the README

- Where read/write routing logic would live (app code, a DB proxy, etc.).
- What replication lag implies, and which operations must always hit the writer.

Evaluation focus

- Understanding of Redis, queues and search in large systems
- Ability to integrate multiple services into a Kubernetes-based platform
- Conceptual grasp of read/write DB clustering and consistency implications
- Isolation and security awareness: platform services stay internal

Task 6 — Observability, Scaling & Rate Limiting

Set up observability and scaling in the local Kubernetes environment to simulate production. Use an `observability/` folder, or add the YAMLs under `k8s/`.

Suggested files

- `prometheus.yaml` or Helm values
- `grafana.yaml` or Helm values
- `fluentbit-daemonset.yaml`
- `loki.yaml` or `es-logging.yaml`
- `hpa-backend.yaml`

Metrics (Prometheus + Grafana)

- Deploy Prometheus to scrape backend pods and/or the ingress controller metrics.
- Deploy Grafana with dashboards showing backend CPU usage, backend memory usage, request rate and error rate, and the number of backend replicas over time.
- Document which metrics you would alert on and why (for example high error rate, high CPU, high latency).

Logging (Fluent Bit + Loki or Elasticsearch)

- Deploy a log collector such as Fluent Bit as a DaemonSet.
- Ship logs to Loki or Elasticsearch.
- Verify that backend logs can be searched and filtered.

Scaling and rate limiting

- Add a HorizontalPodAutoscaler for the backend (`hpa-backend.yaml`).
- Apply rate limiting at the ingress or gateway layer, and explain your choice of limits.

Submission Guidelines

Share a Git repository with cloudlycloud@gmail.com, and make the README the main deliverable.

Repository

- Fork or copy the base repo into your own GitHub, GitLab or Bitbucket account and name it clearly, for example `node-react-postgres-platform-assessment`.
- Work on feature branches per task, then merge into `master`.
- Make small, meaningful commits, such as `task-2: add k8s manifests for dev namespace`, to show your process.
- If the repo is private, add the reviewer as a collaborator, or send a zip as a fallback.

Expected layout

```
.
├── Dockerfile.backend
├── Dockerfile.frontend
├── .dockerignore
├── docker-compose.yml
├── k8s/
├── observability/
├── .github/workflows/ci-cd.yml
├── docs/                # screenshots, diagrams
└── README.md
```

The README must cover

1. Prerequisites and one-command startup for Compose and for kind/minikube.
2. The request path (Browser → Ingress → Service → Pod → PostgreSQL) and how Kubernetes DNS resolves services.
3. The debugging scenarios: Ingress reachable but backend not, and frontend can't reach backend.
4. How CI secrets are stored and used, and your rollback strategy.
5. Read/write DB routing and replication lag notes.
6. Which metrics you would alert on, and why.
7. Your design decisions, trade-offs, and what you would do with more time.

Evidence to put in `docs/`

- Screenshots of `kubectl get all -n dev`
- The app running at `http://node-react.local`
- Grafana dashboards
- Loki log queries
- A passing GitHub Actions run

Evaluation Summary

Each task is reviewed against the focus areas below, and the README is read as the main evidence of your reasoning.

Task	What the reviewer looks for
1. Docker Compose	Multi-stage, minimal-layer Dockerfiles; correct container networking; no hardcoded secrets; clear README startup
2. Kubernetes	Deployments, Services, ConfigMaps, Secrets, PV/PVC, Namespaces; internal vs external separation; probes; readable YAML
3. Ingress / gateway	Ingress vs Service vs Pod model; internal DNS and service discovery; traffic-flow debugging; microservice evolution
4. CI/CD	Structured pipeline; branch-to-environment mapping; secrets handling; safe deployment and rollback
5. Platform services	Redis, queues and search in context; multi-service integration; read/write DB consistency; internal-only isolation
6. Observability	Prometheus and Grafana dashboards; Fluent Bit logging to Loki or Elasticsearch; alert reasoning; HPA; rate limiting